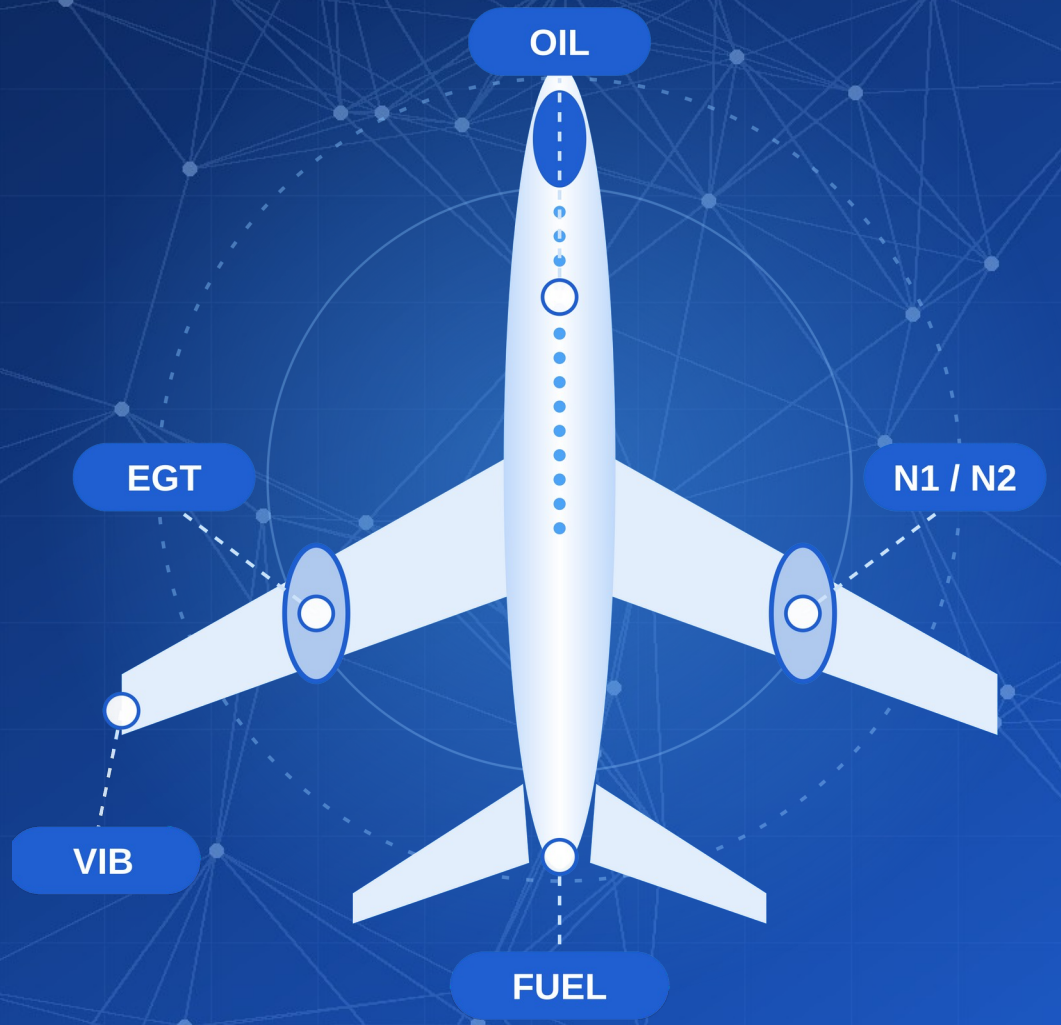
Aircraft Health Classification Using Machine Learning

Predicting Aircraft Health Conditions Using Sensor Data

Algorithm: Decision Tree Classifier

Student Name: [Student Name]
University: [University]
Department: [Department]
Instructor: [Instructor]
Submission Date: [Date]



Project Overview

What the project is about

Classifies aircraft engine health as Normal, Caution or Abnormal from sensor readings using a Decision Tree.

Why health monitoring matters

Early awareness of unusual engine behavior supports safer and better-planned maintenance decisions.

How ML analyzes sensor data

The model learns patterns across 11 input features such as temperature, vibration, pressure and RPM.

Three health categories

Normal

Readings consistent with healthy operation

Caution

Readings that call for closer attention

Abnormal

Readings indicating a potentially abnormal condition

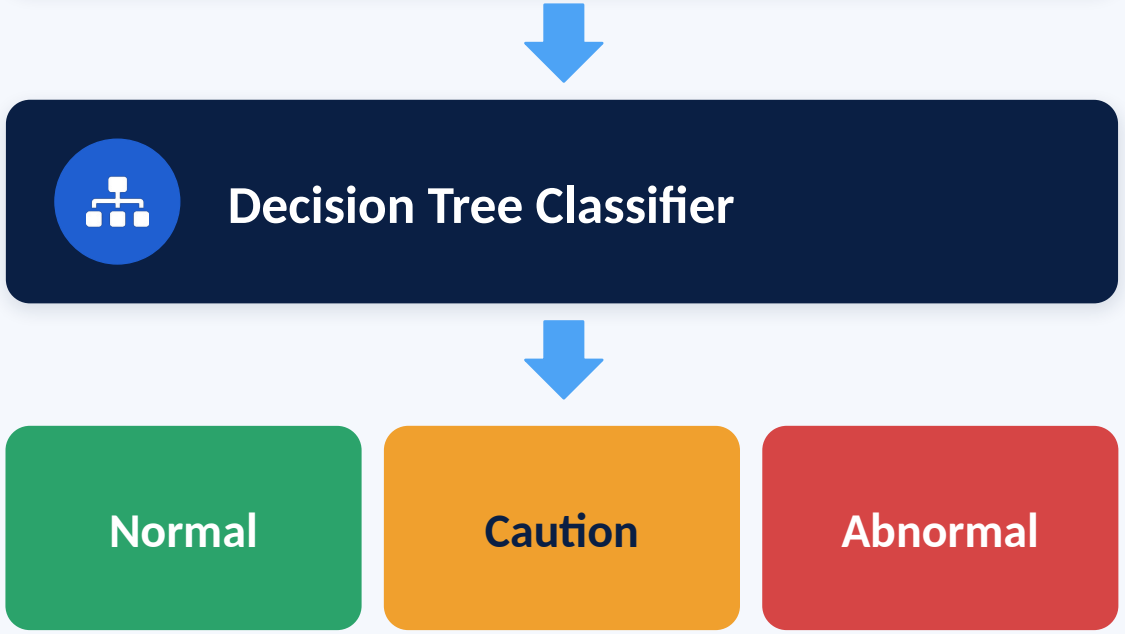
Problem Statement

The challenge

- Engine health depends on many sensor measurements at the same time.
- Checking each reading by hand is slow and hard to scale.
- Patterns across sensors may reveal potentially abnormal operating conditions.
- Goal: classify every record as Normal, Caution or Abnormal.

Input: sensor measurements

Flight hours, oil pressure, N1 and N2 RPM, vibration, fuel pressure, air pressure, fuel flow, exhaust gas temperature, plus operating mode and engine type.



Project Objectives

1

Understand the data

Explore sensor columns and data types

2

Clean and preprocess

Remove duplicates, fill nulls, convert units

3

Explore (EDA)

Summarize distributions and patterns

4

Visualize

Study relationships between sensors

5

Prepare features

Build X and y, encode categories

6

Train the model

Fit a Decision Tree Classifier

7

Evaluate

Accuracy, confusion matrix, report

From raw sensor records to an evaluated health classifier

Understanding the Dataset

11,286
original records

15
original columns

3
duplicate records removed

11,283 × 12
final rows × columns

Flight_Hours
Hr · operating hours

Oil_Pressure
PSI · oil system pressure

Engine_RPM (N1)
% · engine speed N1

Engine_RPM (N2)
% · engine speed N2

Engine_Vibration
Units · vibration level

Fuel_Pressure
PSI · fuel system pressure

Air_Pressure (P2/P3)
PSI · air pressure

Fuel_Flow
PPH · fuel flow rate

Engine_Temperature (EGT)
°C · exhaust gas temperature

Operating_Modes
Categorical · 6 modes + Unknown

Engine_Types
Categorical · 7 types + Unknown

Health_Label
Target · Normal / Caution / Abnormal

Dot colour: royal = numerical (9) · sky = categorical (2) · green = target. Final feature count after dropping Aircraft_ID: 11.

Data Understanding and Inspection

Command	What it shows
`df.head()`	First 5 rows
`df.tail()`	Last 5 rows
`df.shape`	(11286, 16) rows × columns
`df.columns`	Column names
`df.info()`	Data types and non-null counts
`df.describe()`	Summary statistics
`df.isnull().sum()`	Missing values per column
`df.duplicated().sum()`	3 duplicate rows
`df.nunique()`	Unique values per column

Raw structure (df.head(), 3 of 16 columns)

Flight_Hours	Oil_Pressure	Operating_Modes
16761.7 Hr	66.2 PSI	Cruise
4186.1 Hr	71.8 PSI	Climb
3275.3 Hr	57.8 PSI	Cruise

Values were stored as text with units, so they had to be cleaned.

Numerical vs categorical

Numerical (9): flight hours, oil pressure, N1, N2, vibration, fuel pressure, air pressure, fuel flow, EGT

Categorical (2): Operating_Modes, Engine_Types

Target: Health_Label

Data Cleaning and Preprocessing

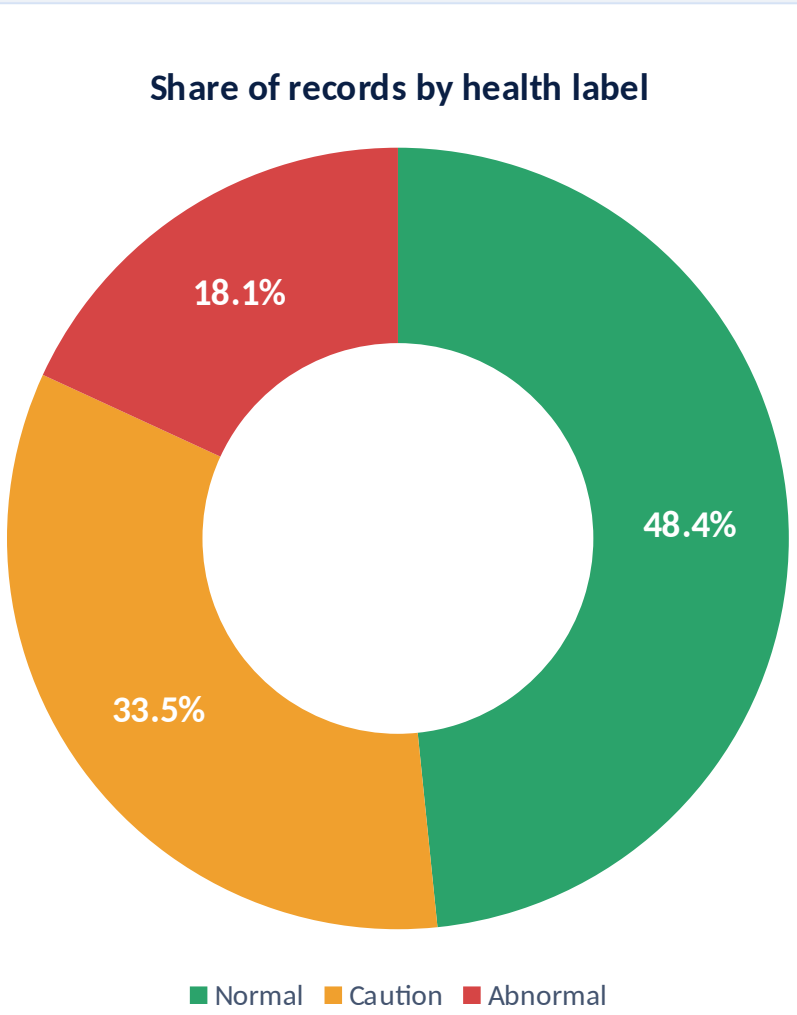
1. Removed unnamed columns (unnamed 1, 2, 3)
2. Removed 3 duplicate records
3. Filled missing Operating_Modes and Engine_Types with Unknown Mode and Unknown Type
4. Renamed Enginerpm (N1) and (N2) to Engine_RPM (N1) and (N2)
5. Removed units (PSI, %, Units, Hr, PPH, °C) from numerical columns
6. Converted measurement columns to float
7. Dropped Aircraft_ID: an identifier, not a sensor feature

Before vs after

Item	Before	After
Shape	11,286 × 16	11,283 × 12
Duplicate rows	3	0
Missing Operating_Modes	8	0
Missing Engine_Types	12	0
Value format	"66.2 PSI" (text)	66.2 (float)
RPM column	Enginerpm (N1)	Engine_RPM (N1)

After cleaning: 11 features + Health_Label = 12 columns

Health Label Distribution



Health Label	Records	Share
Normal	5,459	48.4%
Caution	3,779	33.5%
Abnormal	2,045	18.1%
Total	**11,283**	**100.0%**

Why the target matters

Supervised learning needs known answers. The model learns from Health_Label and is scored against it.

Label logic

Health_Label was added to the dataset. Its exact generation logic should be documented and validated separately.

Exploratory Data Analysis (EDA)

df.describe()

Count, mean, spread, min and max of each numerical column

Histograms

Shape and spread of a single variable, such as Flight_Hours

Bar plots

Averages and counts across categories and health labels

Scatter plots

Relationships between two sensors, coloured by health label

Box plots

Median, quartiles and outliers by health label

Correlation analysis

Strength of linear association between numerical features

EDA reveals: distributions • patterns • relationships between sensors • unusual values

Statistical Analysis

Feature	Mean	Minimum	Maximum
Flight_Hours (Hr)	19,641.22	634.2	60,000
Oil_Pressure (PSI)	55.45	20	92.5
Engine_Vibration (Units)	5.31	0.27	10
Fuel_Pressure (PSI)	42.59	20	69.4
Engine_Temperature (EGT) (°C)	900.52	434	1,100
Engine_RPM (N1) (%)	67.59	15	105
Engine_RPM (N2) (%)	85.37	55	110

Computed on 11,283 cleaned records (df.describe()).

900.52 °C

Mean EGT

85.37 %

Mean N2 RPM

55.45 PSI

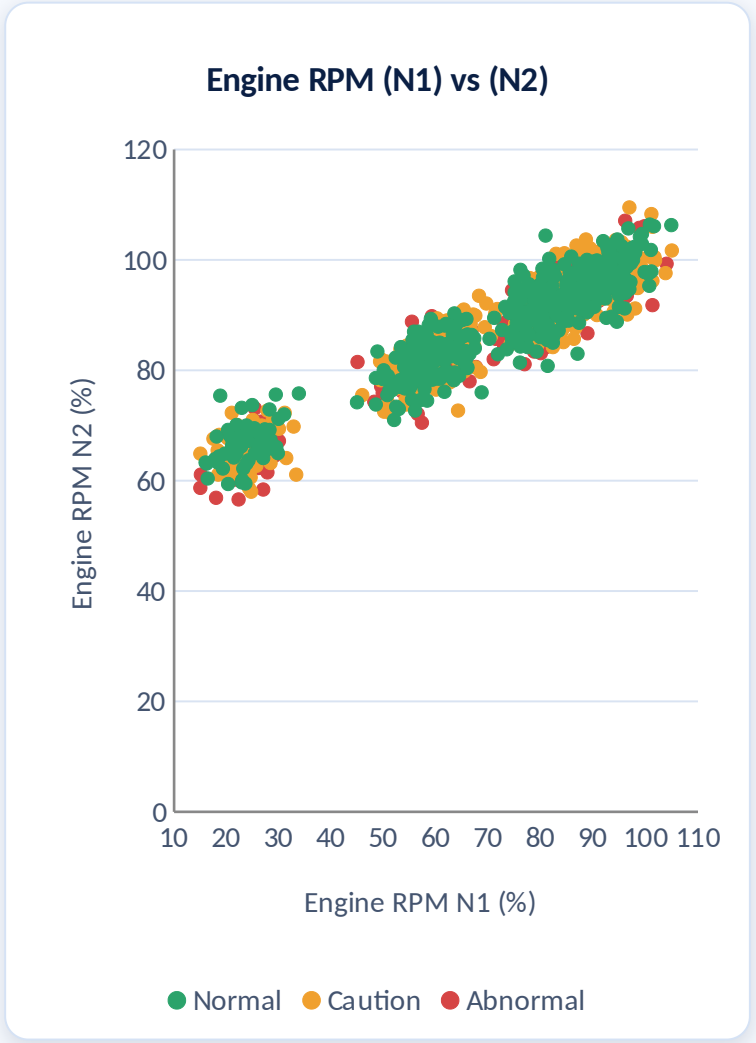
Mean oil pressure

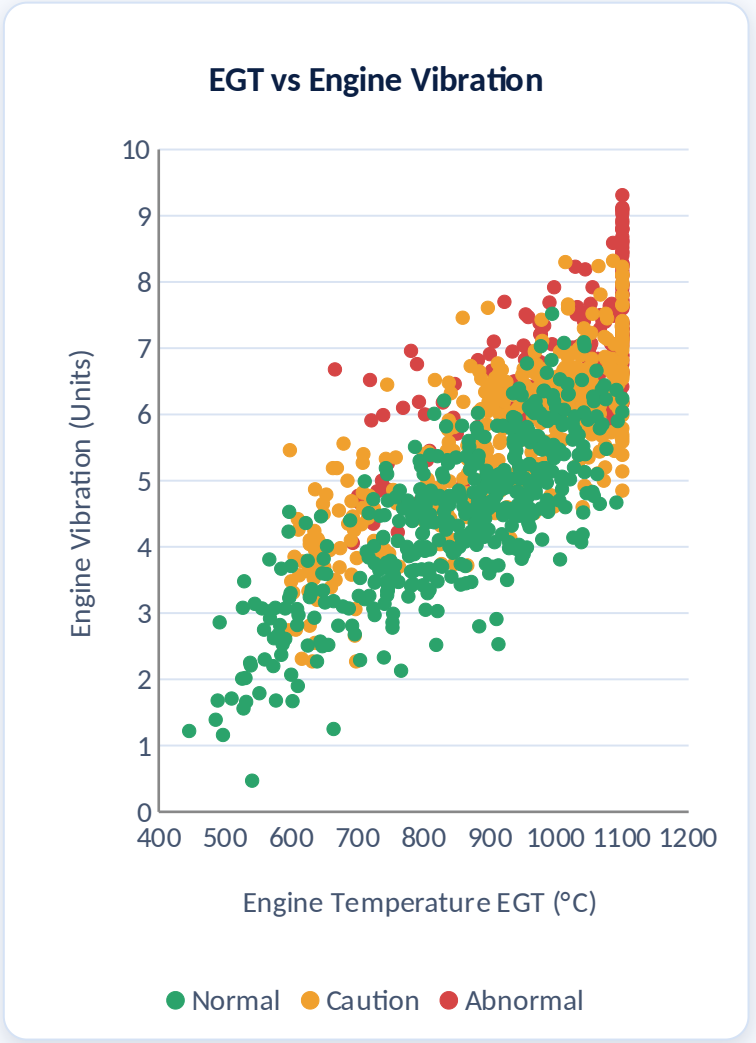
60,000 Hr

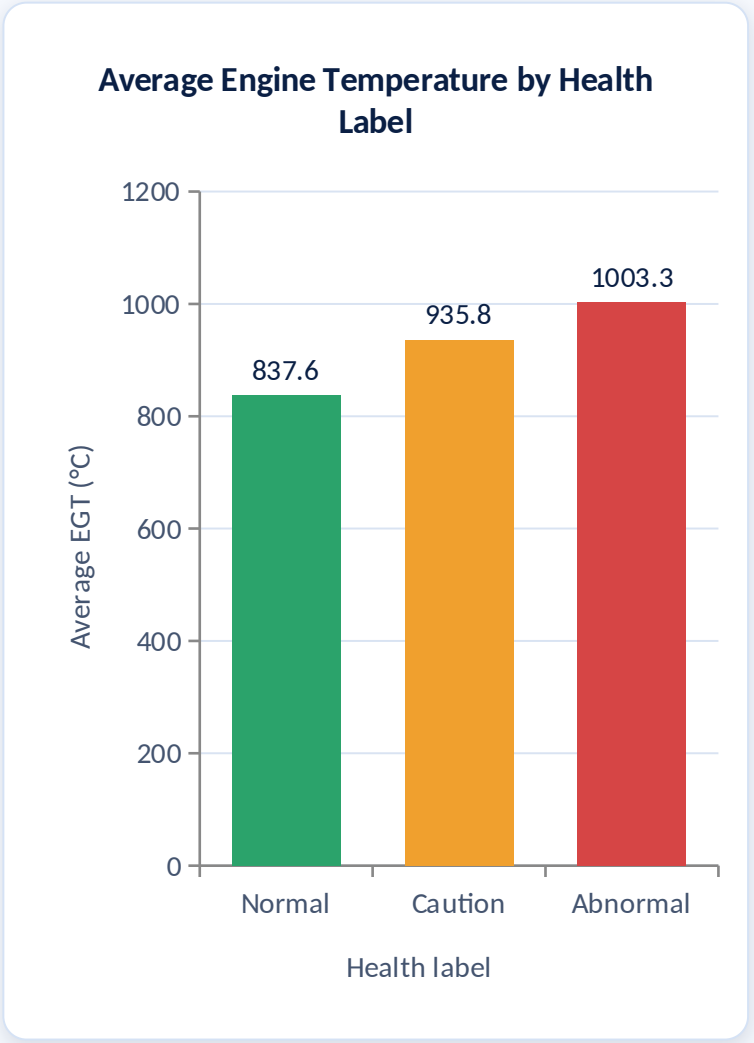
Maximum flight hours

Means shown are averages over all 11,283 records.

Data Visualization: Sensor Relationships

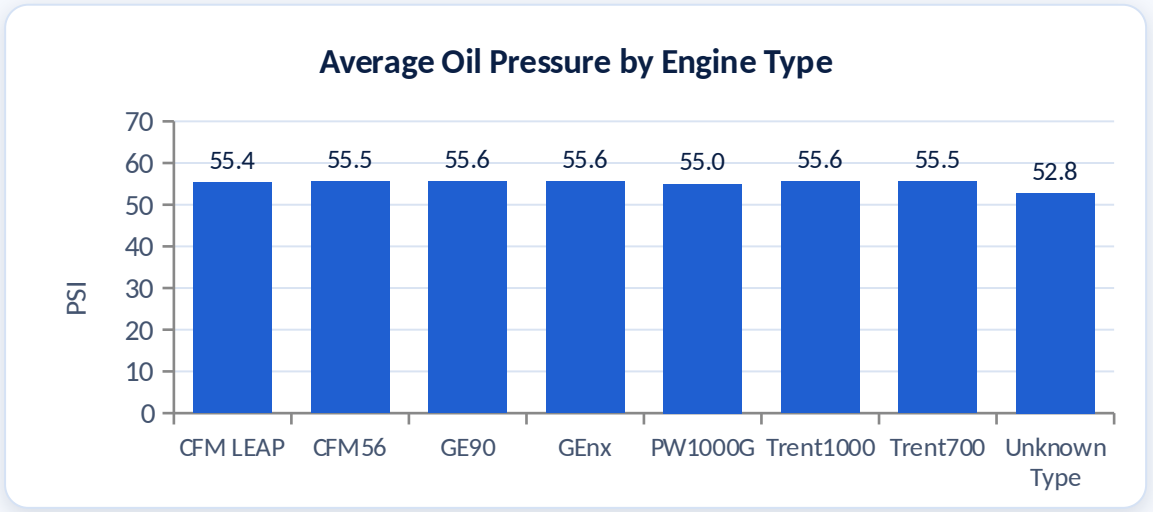


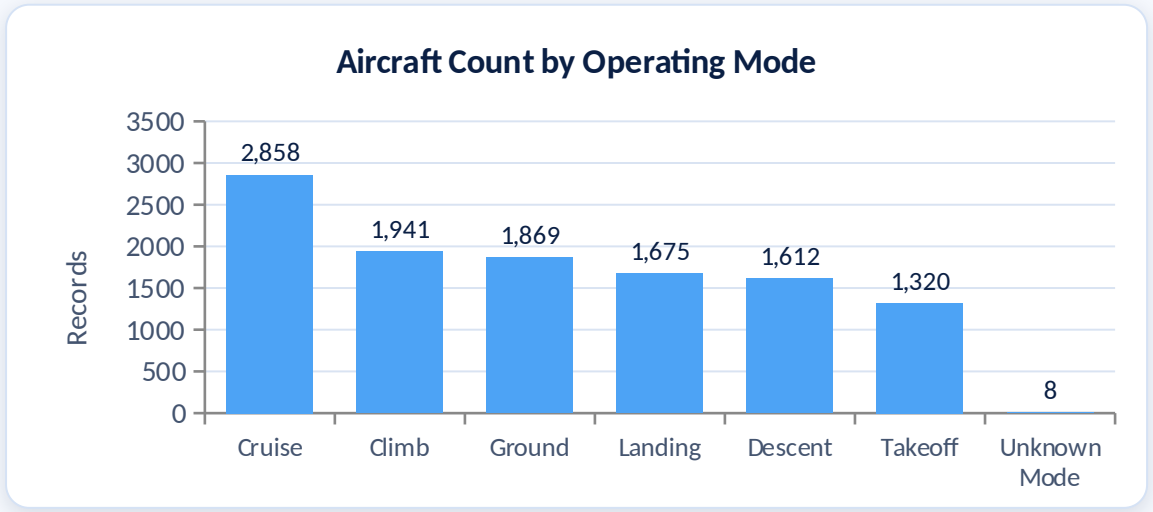


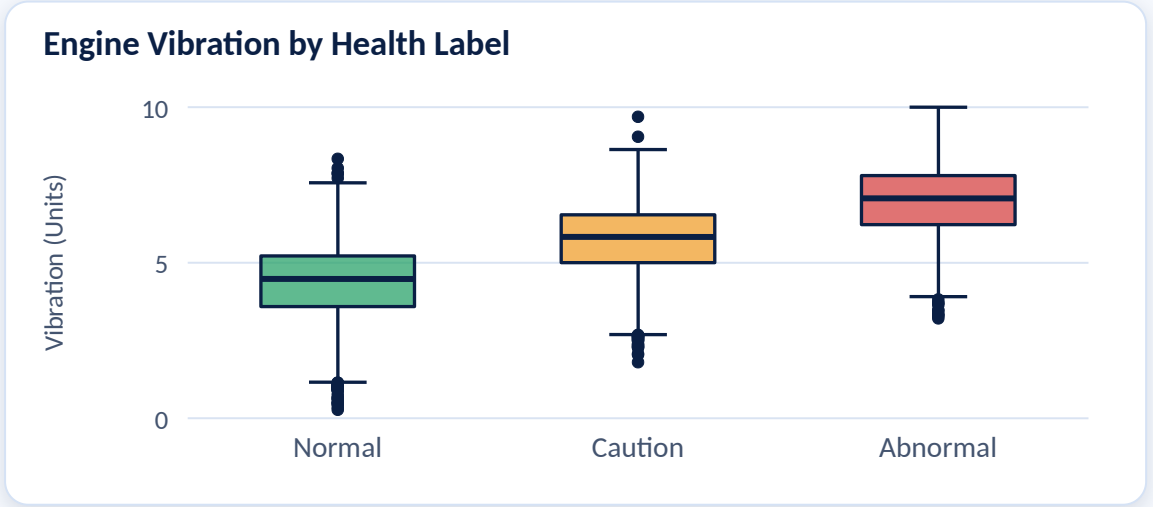


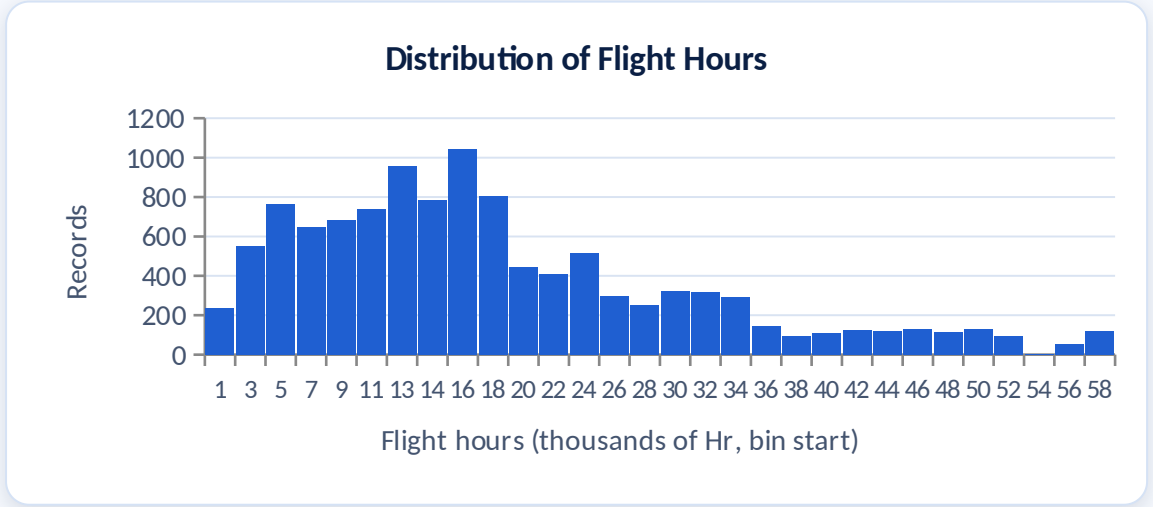
Scatter plots: random sample of 1,000 of 11,283 records. Bar chart: all records.

Data Visualization: Categories and Distributions







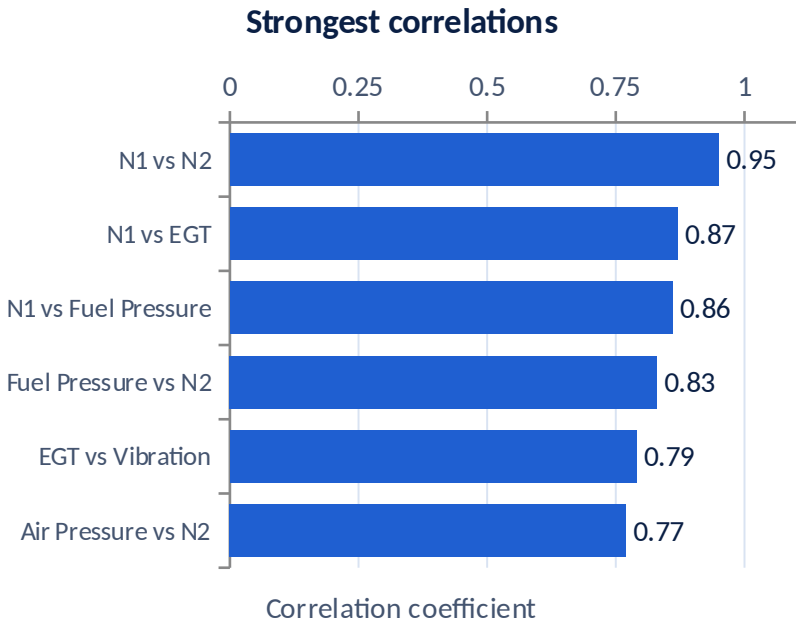


Correlation Analysis

	FltHrs	Oil	N1	Vib	FuelP	AirP	FuelFl	EGT	N2
FltHrs	1.00	-0.07	0.00	0.09	-0.02	-0.05	0.00	0.06	0.00
Oil	-0.07	1.00	0.51	-0.13	0.54	0.66	0.33	0.19	0.56
N1	0.00	0.51	1.00	0.55	0.86	0.71	0.72	0.87	0.95
Vib	0.09	-0.13	0.55	1.00	0.35	0.08	0.44	0.79	0.49
FuelP	-0.02	0.54	0.86	0.35	1.00	0.68	0.61	0.68	0.83
AirP	-0.05	0.66	0.71	0.08	0.68	1.00	0.67	0.43	0.77
FuelFl	0.00	0.33	0.72	0.44	0.61	0.67	1.00	0.66	0.68
EGT	0.06	0.19	0.87	0.79	0.68	0.43	0.66	1.00	0.81
N2	0.00	0.56	0.95	0.49	0.83	0.77	0.68	0.81	1.00

Colour scale: white = 0, blue = 1 (strong positive), pale red = slight negative.

FltHrs = Flight_Hours · Oil = Oil_Pressure · Vib = Engine_Vibration · FuelP = Fuel_Pressure · AirP = Air_Pressure (P2/P3) · FuelFl = Fuel_Flow · EGT = Engine_Temperature



Correlation shows association, not causation.

Feature Selection and Target Variable

Features (X): 11 columns

- Flight_Hours
- Oil_Pressure
- Engine_RPM (N1)
- Engine_RPM (N2)
- Engine_Vibration
- Fuel_Pressure
- Air_Pressure (P2/P3)
- Fuel_Flow
- Engine_Temperature (EGT)

9 numerical

- Operating_Modes
- Engine_Types

2 categorical

Target (y)

Health_Label

- Normal
- Caution
- Abnormal

```
X = df.drop("Health_Label", axis=1)
y = df["Health_Label"]

X.shape  → (11283, 11)
y.shape  → (11283,)
```

Why separate X and y?

The model learns the mapping from inputs (X) to answers (y). Keeping the target out of X prevents the model from simply reading the answer.

Categorical Encoding (One-Hot)

11
features before

24
features after

9 + 15
numerical + encoded

15 encoded columns = 7 + 8

Operating_Modes: 6 modes + Unknown Mode = 7 columns
Engine_Types: 7 engine types + Unknown Type = 8 columns

```
X = pd.get_dummies(
    X,
    columns=["Operating_Modes",
             "Engine_Types"],
    dtype=int
)
```

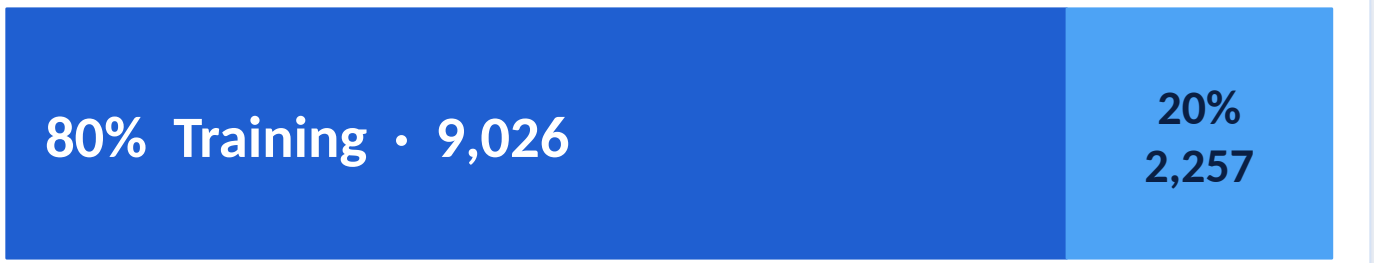
How it works (illustrative example)

Operating_Modes	Cruise	Climb	Landing
Cruise	**1**	0	0
Climb	0	**1**	0
Landing	0	0	**1**

Engine_Types	GEnx	GE90	CFM56
GEnx	**1**	0	0
GE90	0	**1**	0

Each category becomes its own 0/1 column, e.g. Operating_Modes_Cruise and Engine_Types_GEnx, so the tree can use them as numbers.

Train-Test Split (80/20)



Dataset	Records
Training Data	9,026
Testing Data	2,257
Total	11,283

`test_size=0.2`

20% of records held out for testing

`random_state=42`

Same split every time the code runs

`stratify=y`

Class proportions kept in both parts

24

feature columns

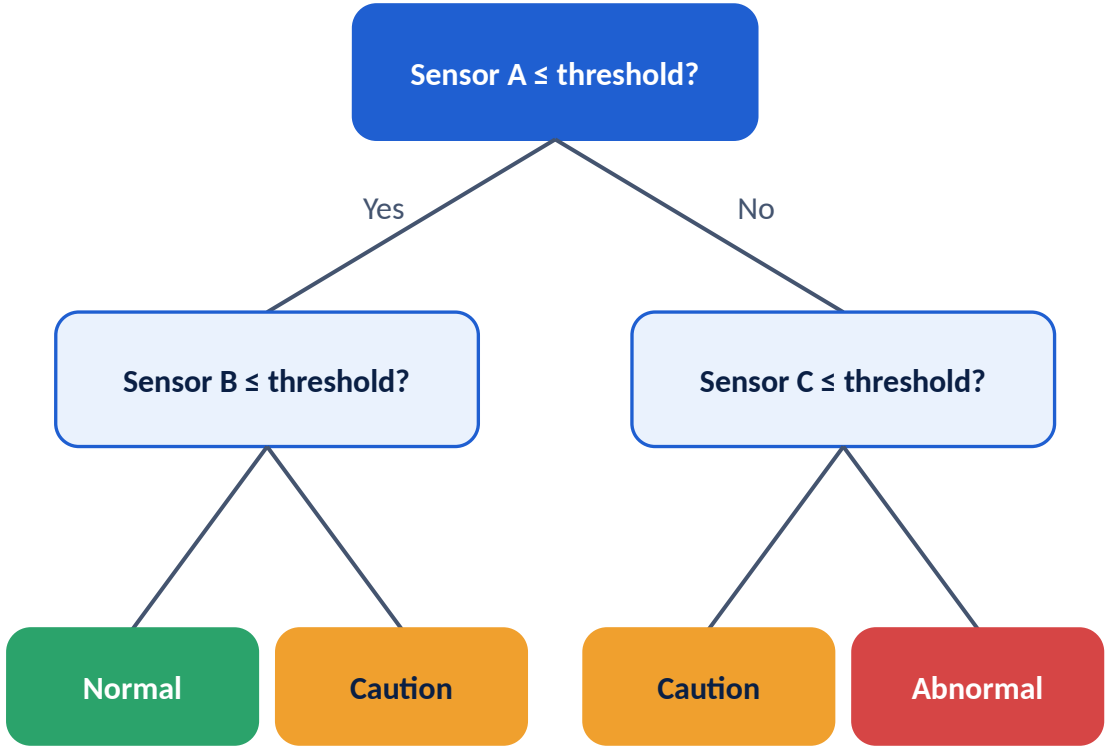
2,257

test records

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    random_state=42, stratify=y
)
```

train_test_split() comes from sklearn.model_selection.

Decision Tree Classifier



Decision nodes ask questions; leaf nodes give the class.

Illustration only: not the trained tree.

```
DecisionTreeClassifier(
    criterion="gini",
    max_depth=10,
    random_state=42
)
```

Gini impurity	How mixed the classes are in a node (lower = purer)
Decision node	Asks a yes/no question about one feature
Splitting feature	The sensor chosen to divide the data
Threshold	The cut-off value tested at a node
Leaf node	Final node that outputs a class
Prediction	The class reached by following the path

Model Training and Prediction




```
model.fit(X_train, y_train)
```

Trains the tree on the 9,026 training records and their labels.

```
y_pred = model.predict(X_test)
```

Predicts a health label for each of the 2,257 test records.

The test records were not used for training.

First 10 test predictions

Actual	Predicted
Caution	Caution
Normal	Normal
Caution	Caution
Abnormal	Abnormal
Normal	Normal
Abnormal	Abnormal
Caution	Caution
Normal	Normal
Normal	Normal
Normal	Normal

Accuracy Score

78.64%

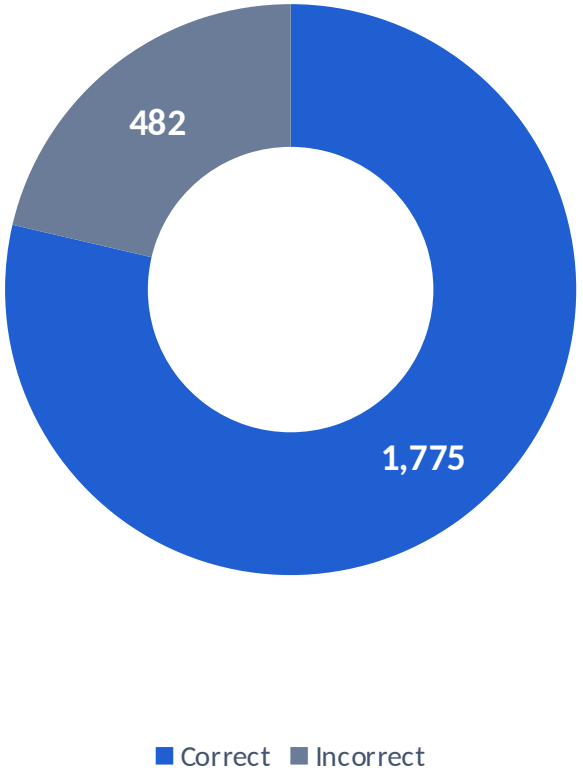
Test accuracy

How it is calculated

Accuracy = correct predictions ÷ total test records

1,775 ÷ 2,257 = 78.64%

Test predictions

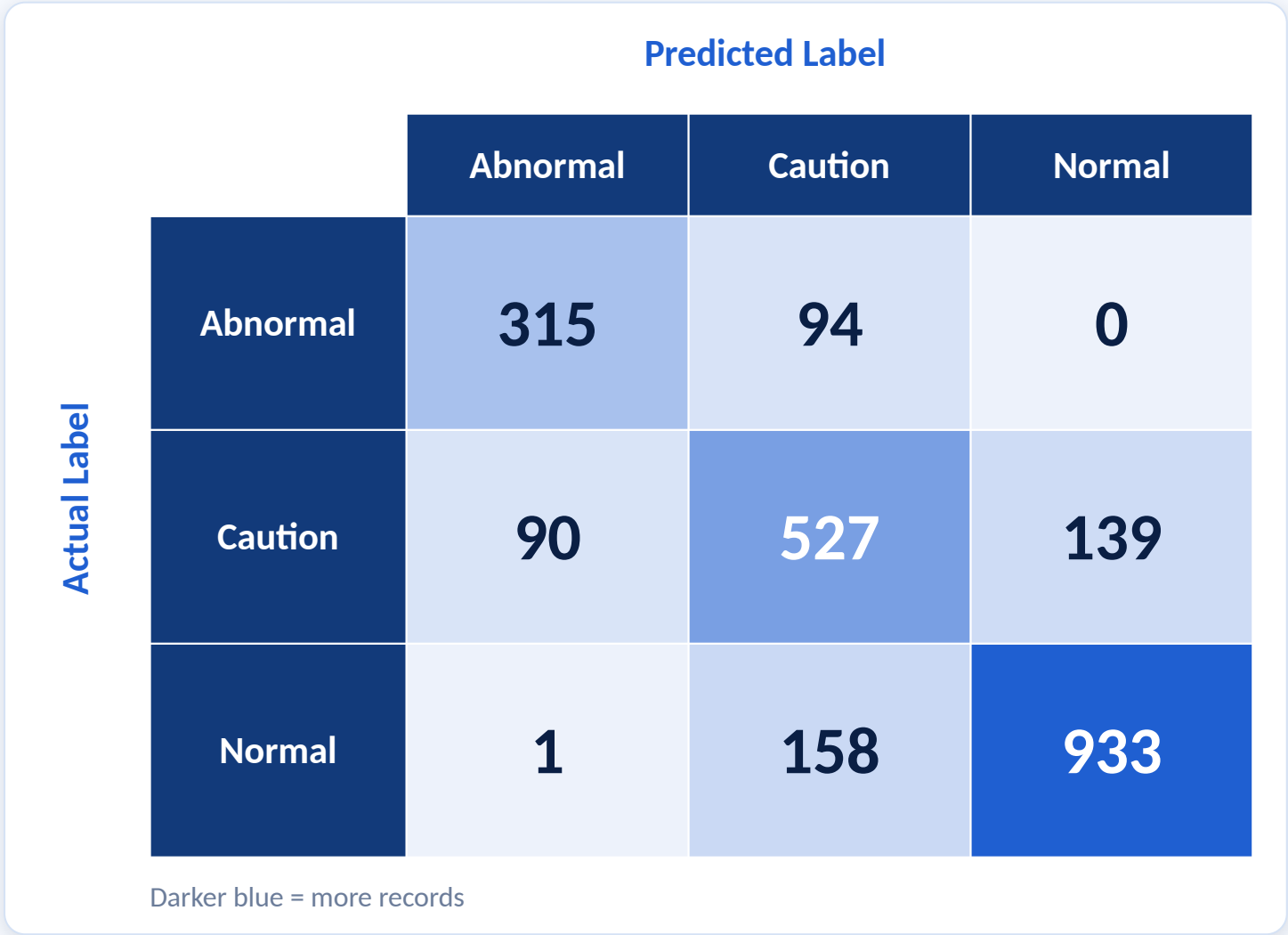


Correct predictions per class

Class	Correct	Test
Abnormal	315	409
Caution	527	756
Normal	933	1,092
Total	**1,775**	**2,257**

Accuracy measures the share of test records predicted correctly. It does not show which classes are confused; see the confusion matrix.

Confusion Matrix



1,775

correct predictions (diagonal)

482

incorrect predictions (off-diagonal)

Diagonal values are correct predictions. Off-diagonal values are errors. 481 of the 482 errors involve the Caution class.

Classification Report

Class	Precision	Recall	F1-Score	Support
Abnormal	0.78	0.77	0.77	409
Caution	0.68	0.70	0.69	756
Normal	0.87	0.85	0.86	1,092
Accuracy			**0.79**	2,257
Macro Avg	0.77	0.77	0.77	2,257
Weighted Avg	0.79	0.79	0.79	2,257

Precision: share of predicted labels that were right. Recall: share of actual records the model found. F1: balance of both.

0.79
accuracy (test set)

0.77
macro average F1-score

0.79
weighted average F1-score

Support = number of test records.

Conclusion and Limitations

Conclusion

- Prepared the aircraft sensor dataset (11,283 × 12).
- Performed EDA and visualization.
- Converted categorical features to numerical (24 features).
- Trained a Decision Tree Classifier.
- Achieved 78.64% test accuracy.
- Evaluated with a confusion matrix and classification report.

Limitations

- Predictions depend on the quality and reliability of the labels.
- If labels came from sensor-based rules, the model may learn those rules, not verified aircraft health.
- The dataset is synthetically generated, not recorded from real aircraft.
- Academic project, not a certified aviation safety or maintenance system.

Thank You

Questions and discussion welcome